UCLA Samueli School of Engineering

CS-M151B
Computer Systems Architecture

Blaise Tine
UCLA Computer Science

Recap: Out of Order Execution

- OoO Design Challenges
  - How do we resolve data dependencies?
  - How do we handle pipeline hazards?
  - How do we preserve ordering for external states?

Recap: Resolving Data Dependencies

Register renaming eliminates

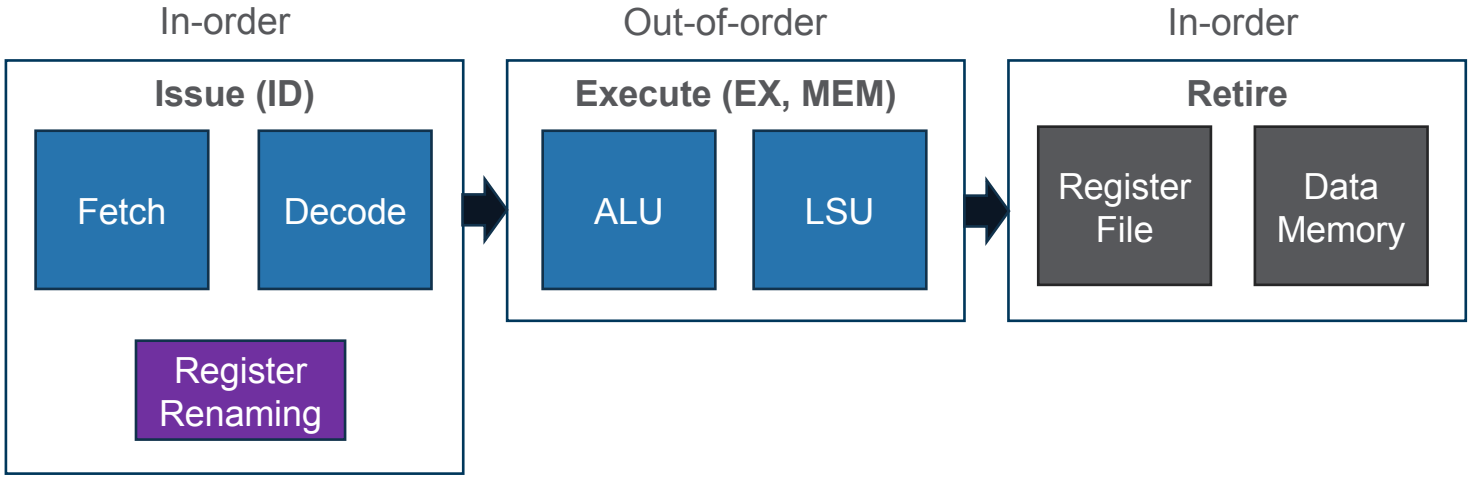
- Resolve WAR dependencies
- Resolve WAW dependencies
- Doesn't resolve RAW dependencies

Program code

```
I1: ADD R1, R3, R3
I2: SUB R2, R1, R5
I3: AND S, R11, R7
I4: OR  R8, S,  R6
I5: XOR T,  R4, R11
```

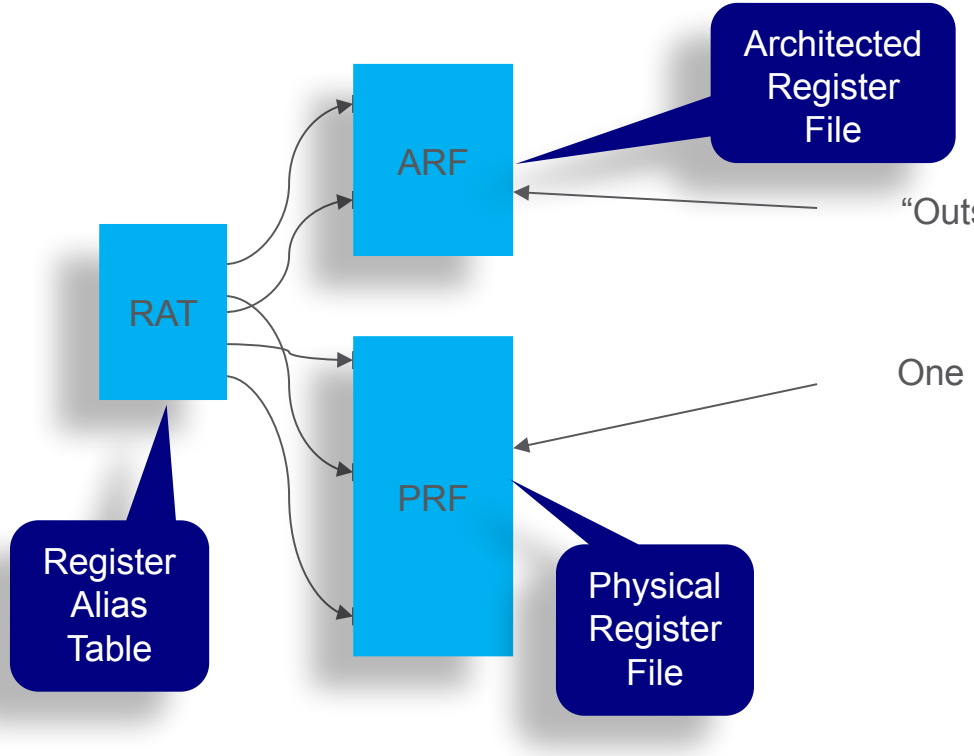
Register Renaming

- Resolving Data dependencies



Recap: Renaming Architecture

- RAT
- PRF
- ARF

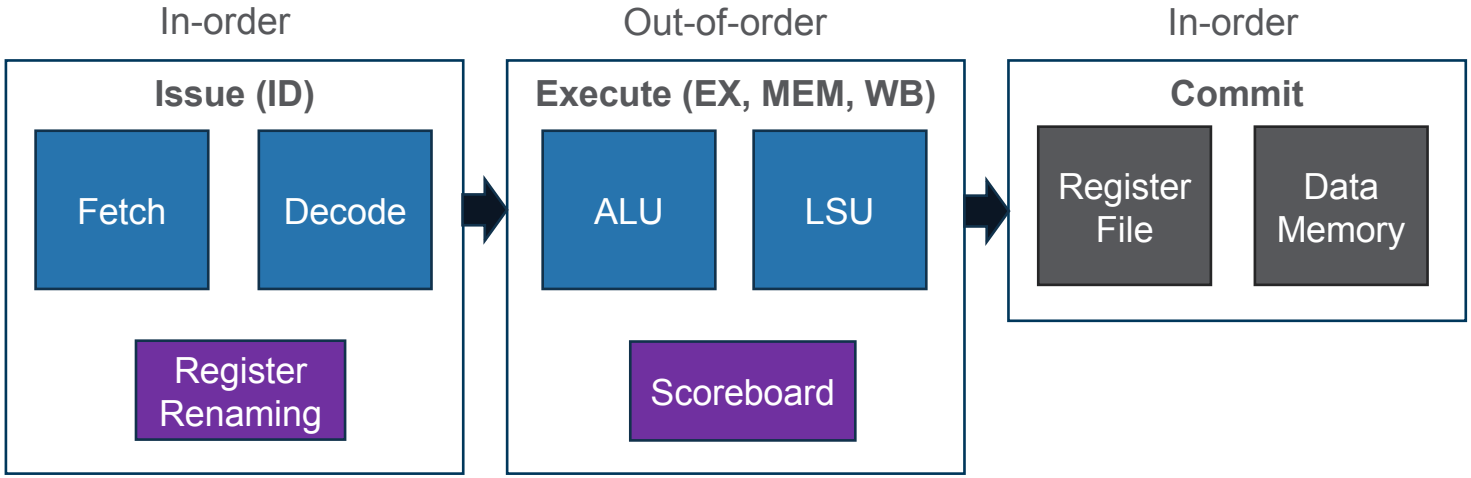


"Outside" world sees the ARF

One PREG per instruction in-flight

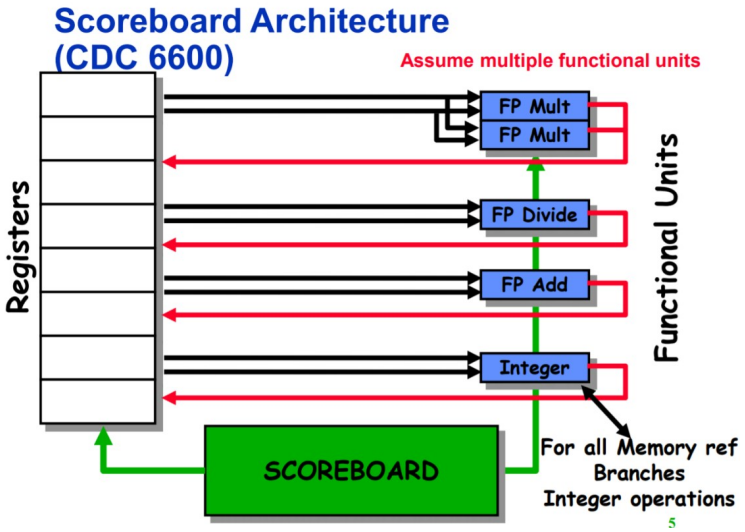
Recap: Scoreboarding

- Resolving RAW dependencies



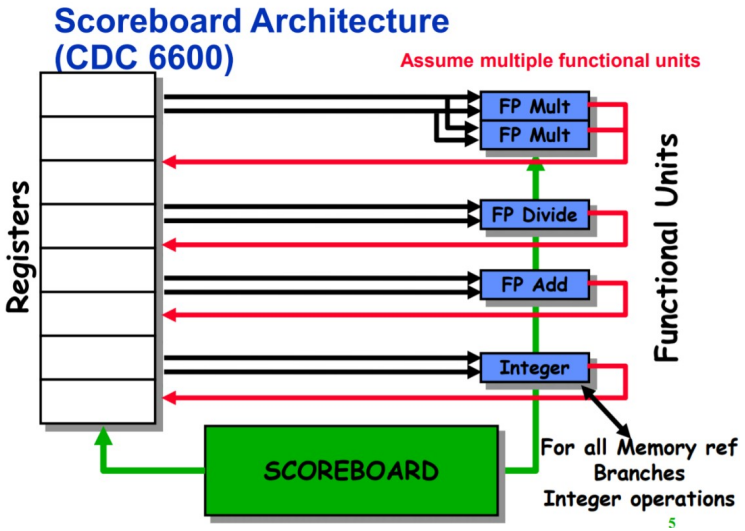
Recap: CDC6600 Scoreboarding

- Architecture
  - 4 stages: IS→RO→EX→WB
  - Scoreboard
  - Register status table (RST)
- Data Hazards
  - WAW, WAR: Stall
  - RAW: Scoreboarding
- Control/Structural Hazards
  - Stall



Recap: CDC6600 Scoreboarding

- In-order completion
  - Not enforced!
  - No h/w exceptions



https://hackmd.io/@a5180352/S1FFbP6pz?type=view

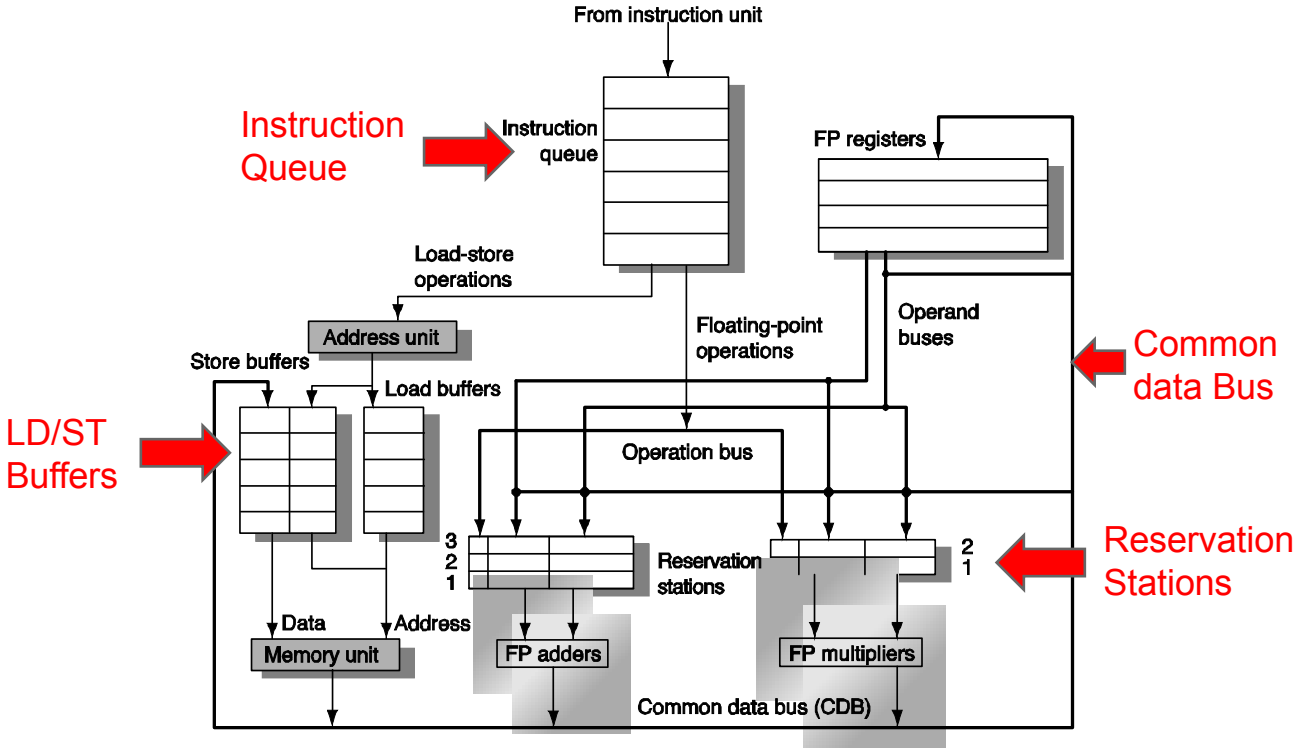
Dynamic Scheduling

Tomasulo's Algorithm

- By Robert Tomasulo and used in IBM 360 in the 1967
- Tracks when operands are available to satisfy data dependences
- Removes name dependences through register renaming
- Very similar to what is used today
  - Almost all modern high-performance processors use a derivative of Tomasulo's… much of the terminology survives to today.

https://hackmd.io/@a5180352/S1FFbP6pz?type=view

Dynamic Scheduling



Three Stages of Tomasulo Algorithm

Issue—get instruction from Instruction Queue

Execution—operate on operands (EX)

Write result—finish execution (WB)

https://hackmd.io/@a5180352/S1FFbP6pz?type=view

Issue Stage

- Get next instruction from instruction queue.
- Find a free *reservation station* for it
  (if none are free, stall until one is)
- Read operands that are in the registers
- If the operand is not in the register,
  find which reservation station will produce it
- **In effect, this step renames registers**
  (reservation station IDs are "temporary" names)

https://hackmd.io/@a5180352/S1FFbP6pz?type=view

Issue Stage

Instruction Buffers

0.	F2 = F4 + F1
1.	F1 = F2 / F3
2.	F4 = F1 – F2
3.	F1 = F2 + F3

To-Do list (from last slide):
Get next inst from IB's
Find free reservation station
Read operands from RF
Record source of other operands
Update source mapping (RAT)

Reg File

F1	3.141593
F2	-1.00000
F3	2.718282
F4	0.707107

RAT

F1	0
F2	α
F3	0
F4	0

Reservation stations

Adder

A1 (α)	F2=F4+F1	0.7071	3.14
A2 (β)			
A3 (χ)			

FP-Cmplx

C1 (δ)	F1 = F2/F3	α(A1)	2.718282
C2 (ε)			

Execute

- Monitor results as they are produced
- Broadcast result to all reservation stations with operands waiting for it (via common data bus)
- When *all operands available* for an instruction, it is ready for execution.
- When multiple instructions in RS are ready?
  - Pick any
  - Except for load/store (must be ordered to avoid memory hazards)

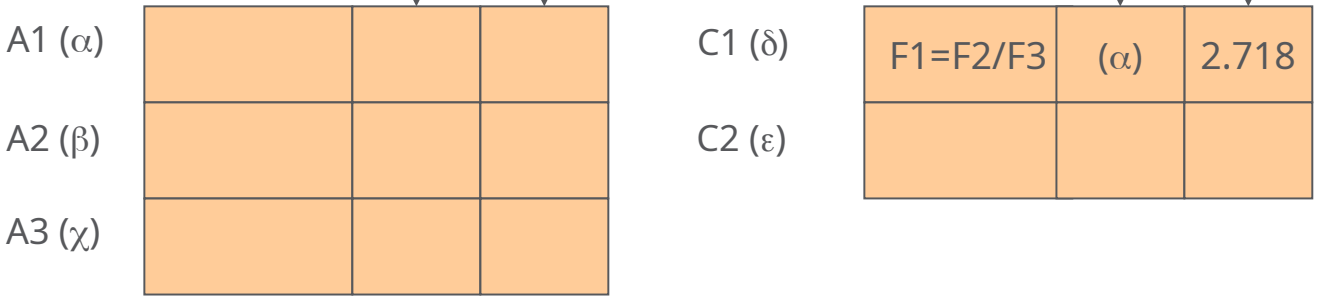
Execute

0. F2 = F4 + F1
1. F1 = F2 / F3
2. F4 = F1 – F2
3. F1 = F2 + F3

F2=F4+F1

(α) 3.8487

To-Do list (from last slide):
Monitor results from ALUs
Capture matching operands
Compete for ALUs



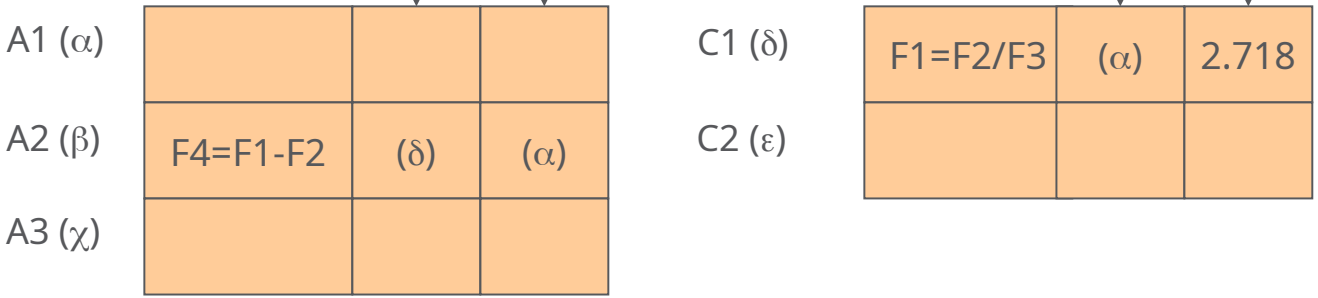
Execute

0.	F2 = F4 + F1
1.	F1 = F2 / F3
2.	F4 = F1 – F2
3.	F1 = F2 + F3

F2=F4+F1

(α) 3.8487

To-Do list (from last slide):
Monitor results from ALUs
Capture matching operands
Compete for ALUs



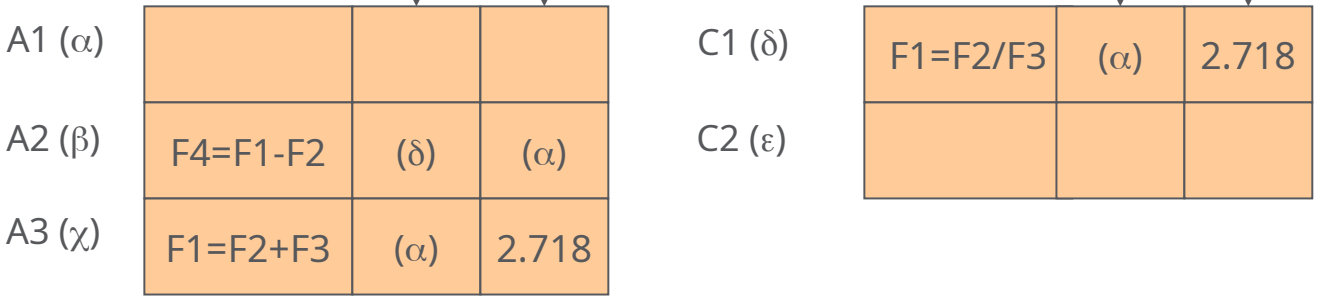
Execute

0.	F2 = F4 + F1
1.	F1 = F2 / F3
2.	F4 = F1 – F2
3.	F1 = F2 + F3

F2=F4+F1

(α) 3.8487

To-Do list (from last slide):
Monitor results from ALUs
Capture matching operands
Compete for ALUs



Execute

0.	F2 = F4 + F1
1.	F1 = F2 / F3
2.	F4 = F1 – F2
3.	F1 = F2 + F3

F2=F4+F1

(α) 3.8487

To-Do list (from last slide):
Monitor results from ALUs
Capture matching operands
Compete for ALUs

A1 (α)			
A2 (β)	F4=F1-F2	(δ)	3.8487
A3 (χ)	F1=F2+F3	3.8487	2.718

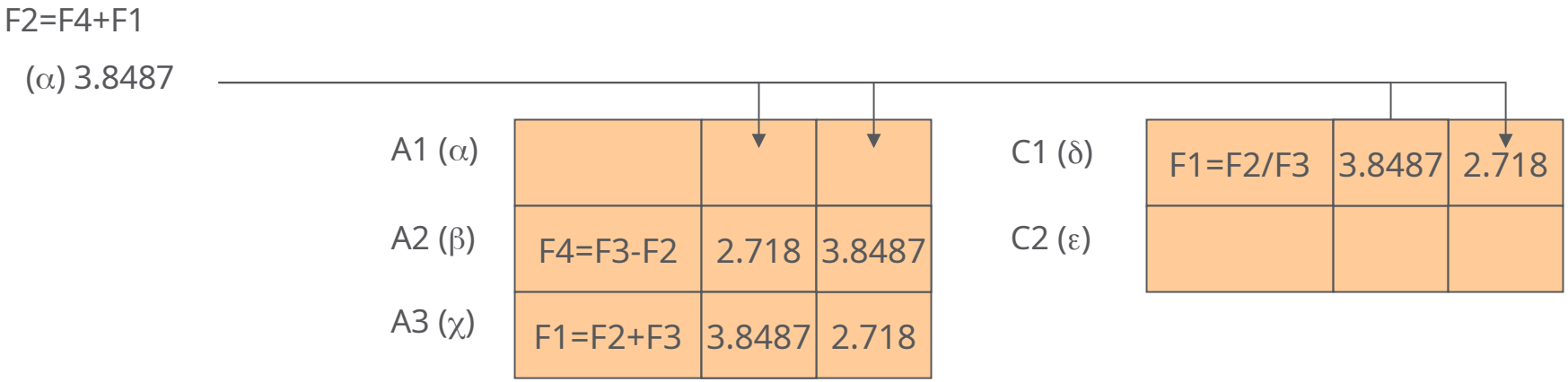
Adder

C1 (δ)	F1=F2/F3	3.8487	2.718
C2 (ε)			

FP-Cmplx

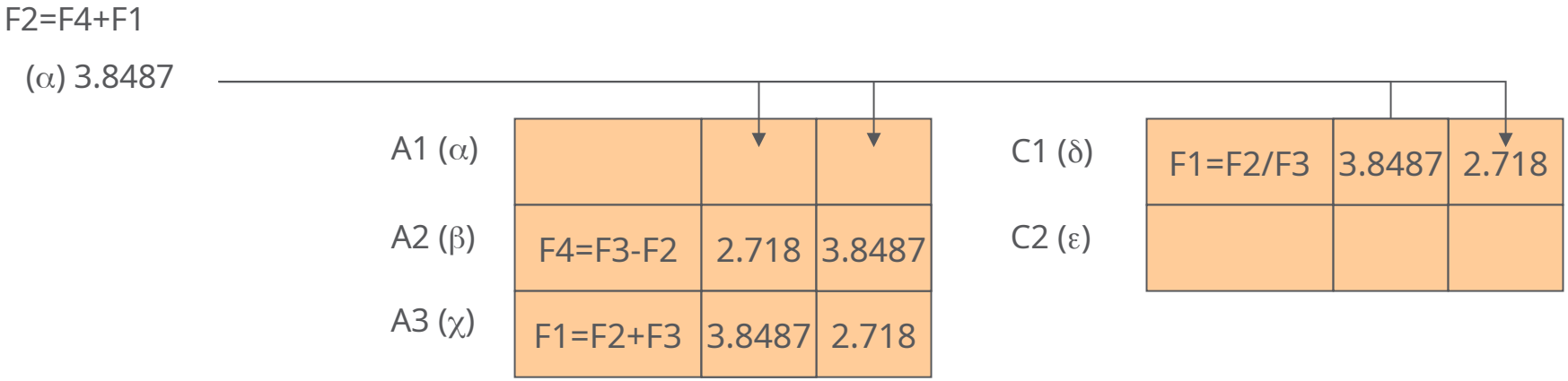
Execute

- Broadcast source priority?



Execute

- Broadcast source priority? Any
- Cheapest: priority encoder



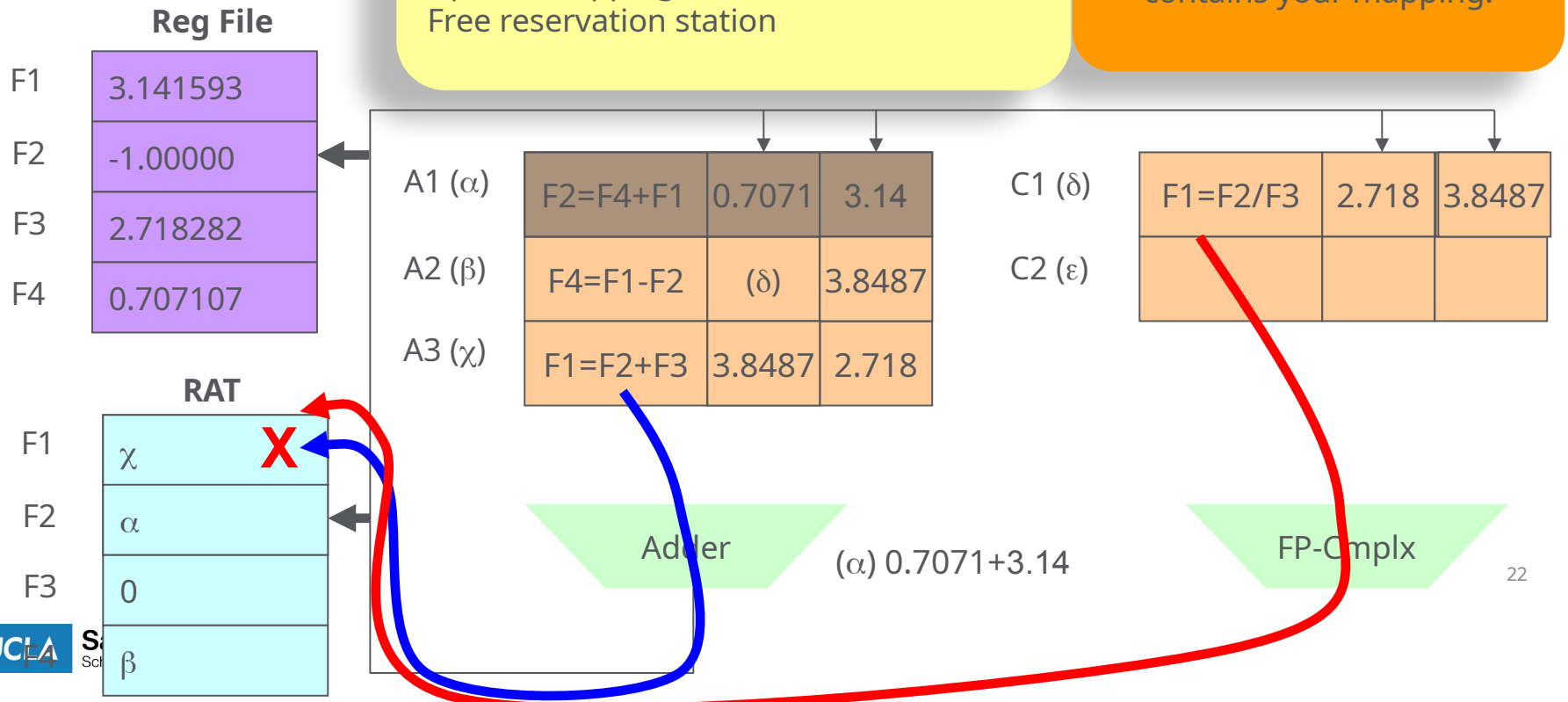
Write Result

- When result is computed, make it available on the "common data bus" (CDB), where waiting reservation stations can pick it up
- Stores write to memory
- Result stored in the register file
- This step frees the reservation station
- For our register renaming, this recycles the temporary name (future instructions can again find the value in the actual register, until it is renamed again)

Write Result

To-Do list (from last slide):
Broadcast on CDB
Writeback to RF
Update Mapping
Free reservation station

Only update RAT (and RF) if RAT still contains your mapping!



Tomasulo's Algorithm: Load/Store

- The reservation stations take care of dependencies through registers.
- Dependences also possible through memory
  - Loads and stores not reordered in original IBM 360
  - We'll talk about how to do load-store reordering later

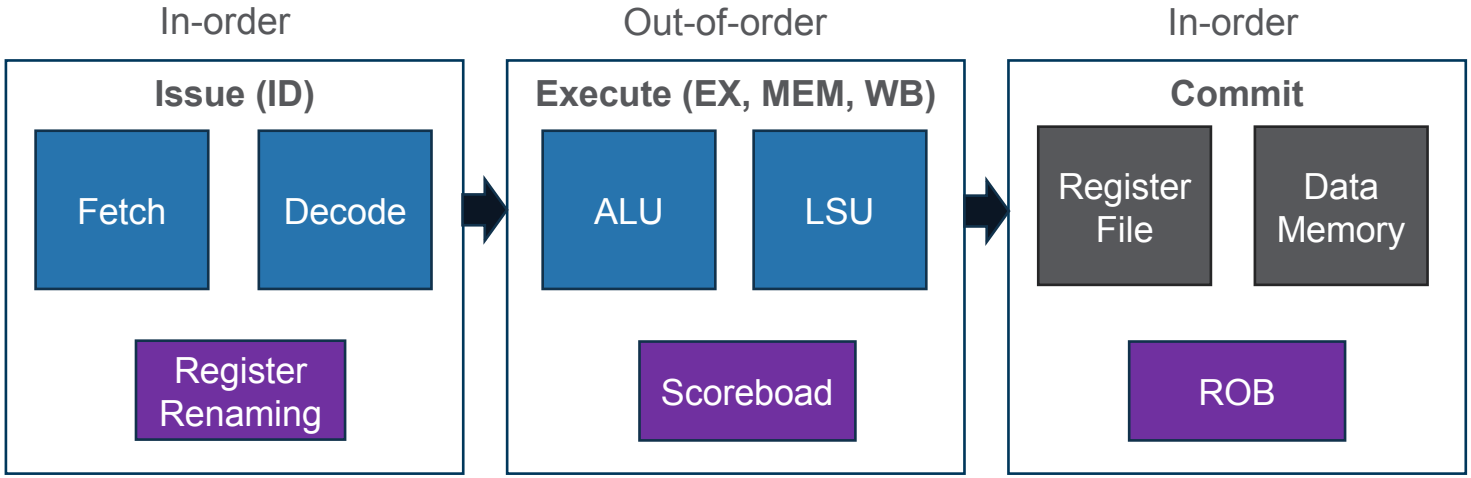
OoO Execution

- We're now executing instructions in data-flow order
  - RAW, WAR, WAW: Ok
  - Great! More performance

- But outside world can't know about this
  - How will you handle debugging in S/W for instance? Exceptions/interrupts?
  - Must maintain illusion of sequentially

Re-order Buffer (ROB)

- Reordering instructions retirement



Re-order Buffer

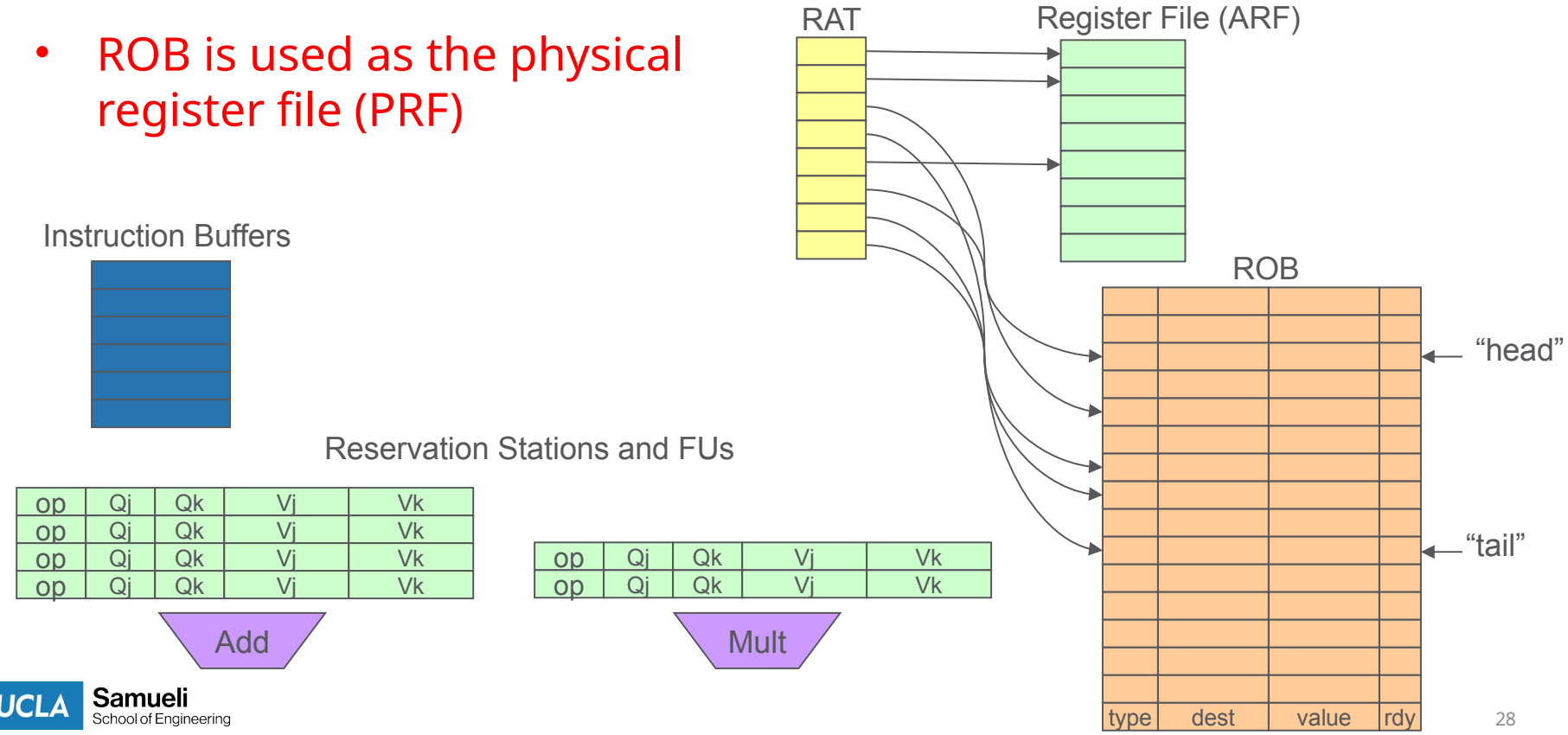
- Motivations
  - Maintaining program order semantics
    - Ensuring that CPU resources accessible to the application are updated in correct order – (e.g. code debugging)
  - Facilitate precise exception handling
    - Ensuring that only the effects of instructions up to the one causing the exception are committed
  - Enabling Speculative Execution
    - Supporting the ability to discard the result of speculatively executed instructions (e.g. branch misprediction)

Re-order Buffer

- ROB operations
  - Separate architected (ARF) vs physical registers (PRF)
  - Track program order of all in-flight instructions
  - Enable in-order completion / commit / retirement
- ROB entries
  - Type: ALU, FPU, LD, ST
  - Dest: destination register to update RAT
  - Value: written value from CDB
  - Ready: value available

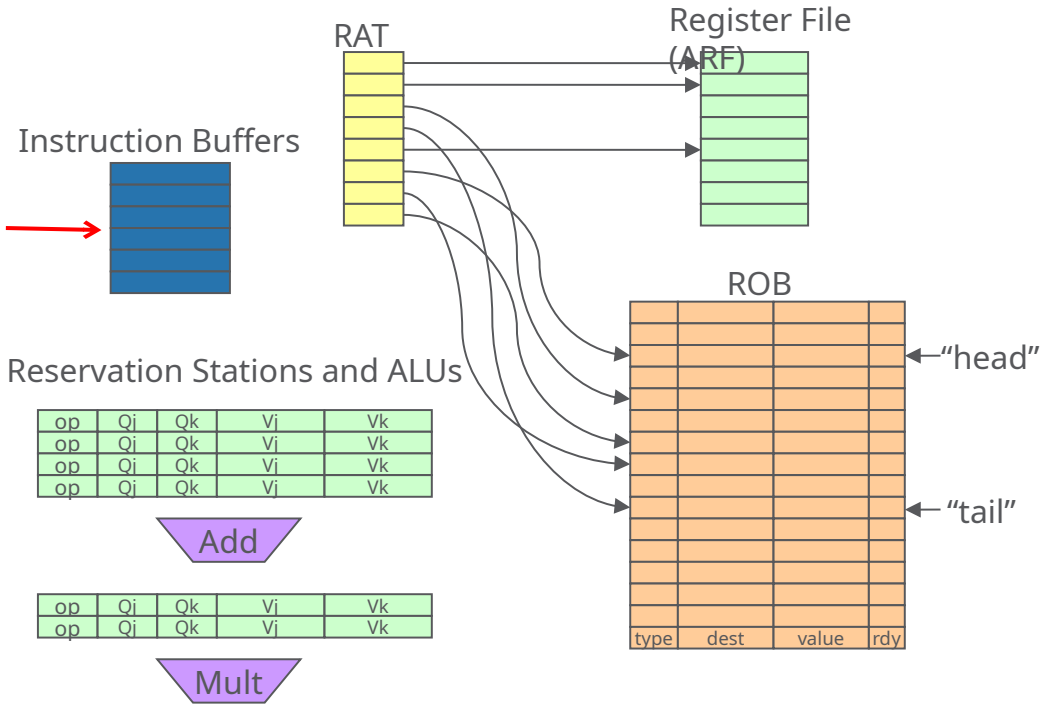
Hardware Organization

- ROB is used as the physical register file (PRF)



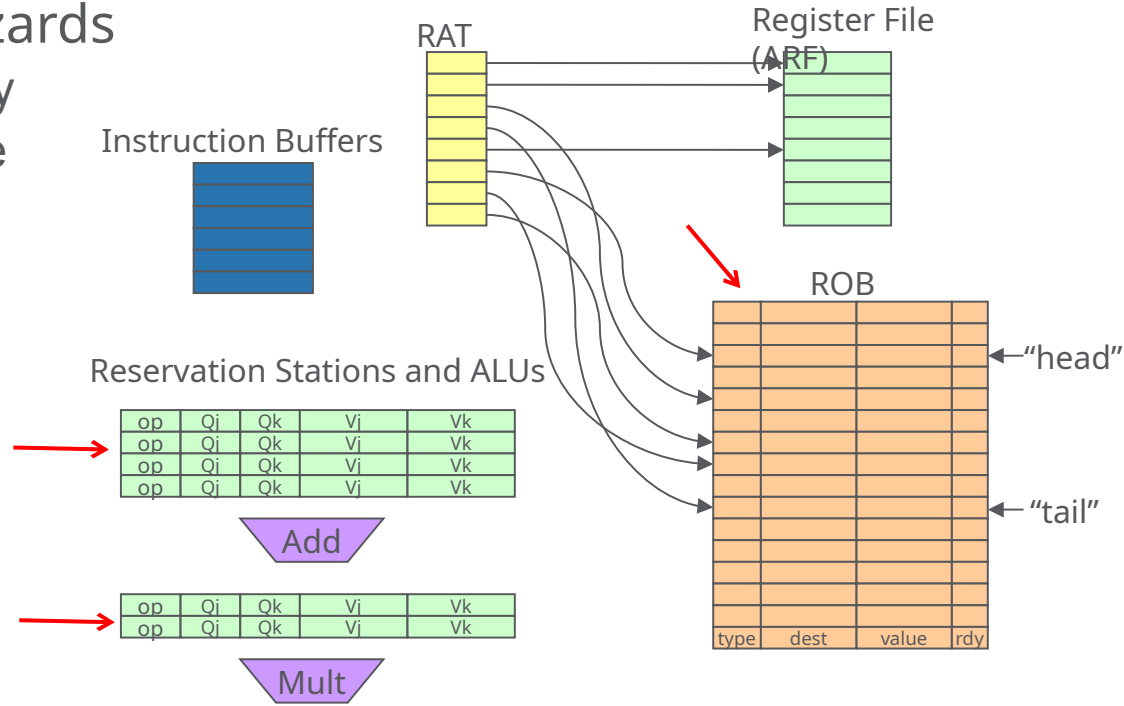
Issue Stage

- Read next instruction from IBuffer



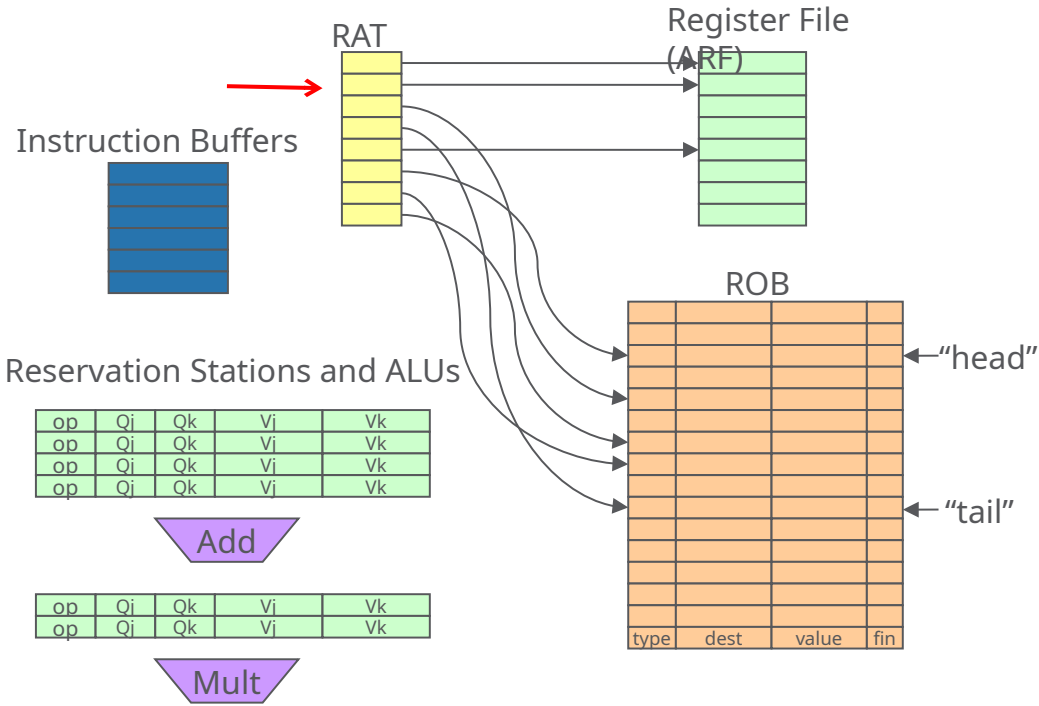
Issue Stage (2)

- Check for structural hazards
  - RS and ROB availability
  - Stall if any not available



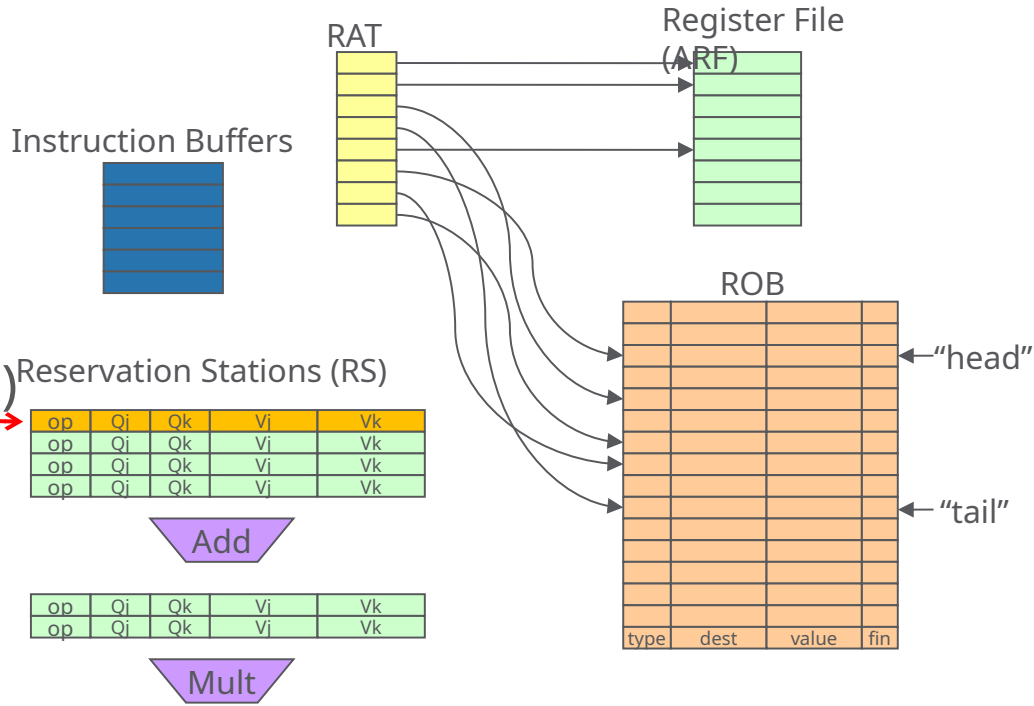
Issue Stage (3)

- Read the RAT
  - Find the corresponding PRF registers given the instruction source registers
  - PRF registers are ROB entries
  - Store dest ROB index in RS
  - Store src ROB indices in RS



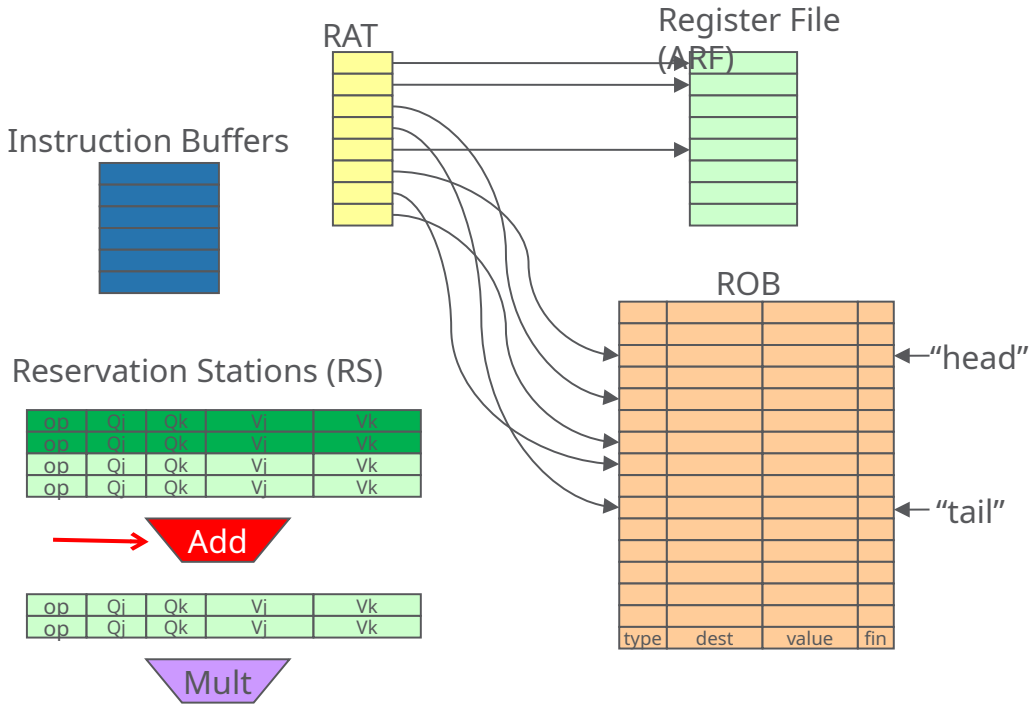
Execute Stage

- Busy reservation stations have issued instructions
- Each wait for all source operands to be valid
- Operands are updated via the common data bus (CDB)



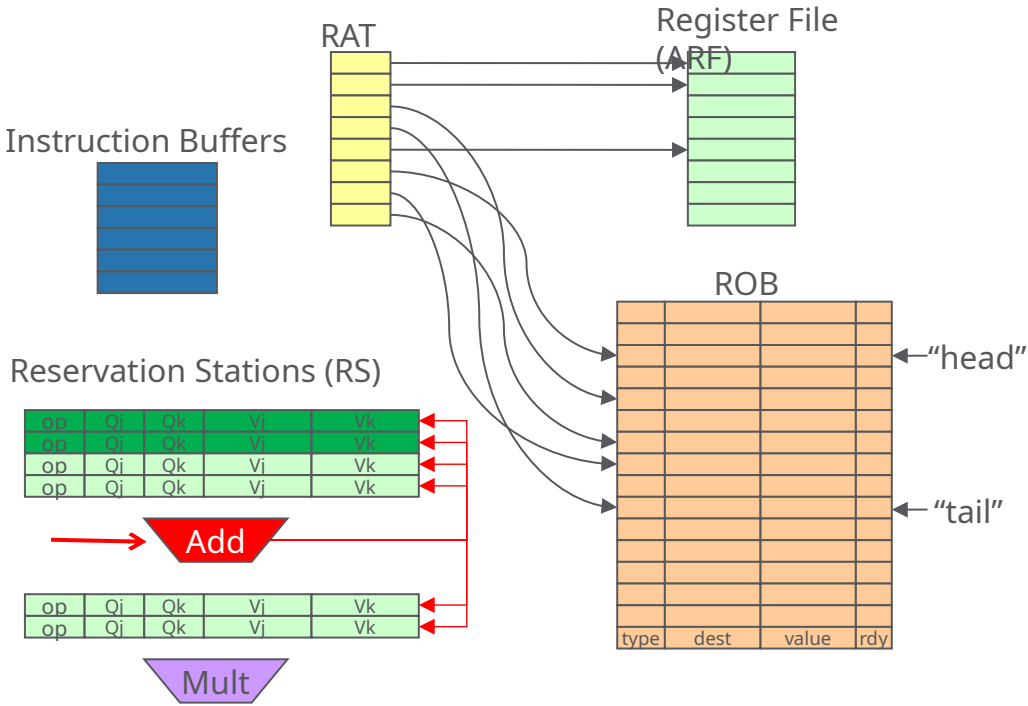
Execute Stage (2)

- One RS is assigned a functional unit (FU)
- Execution proceeds



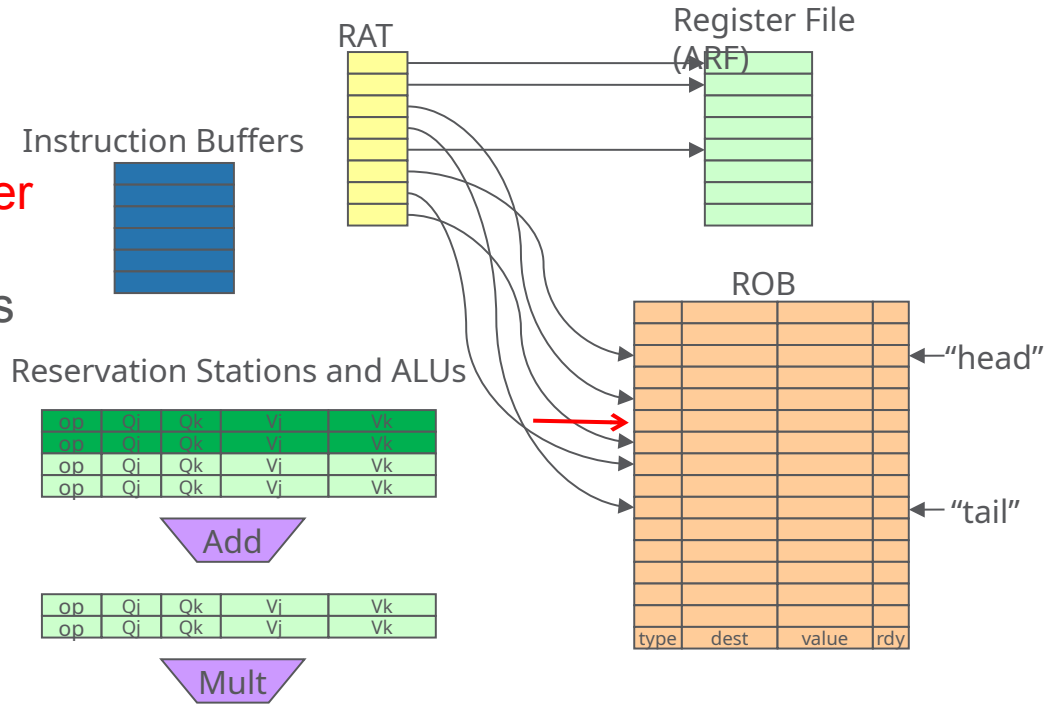
Write Result

- Broadcast result on CDB
- Update dependent RS source operands
- Do not update register file (ARF)!



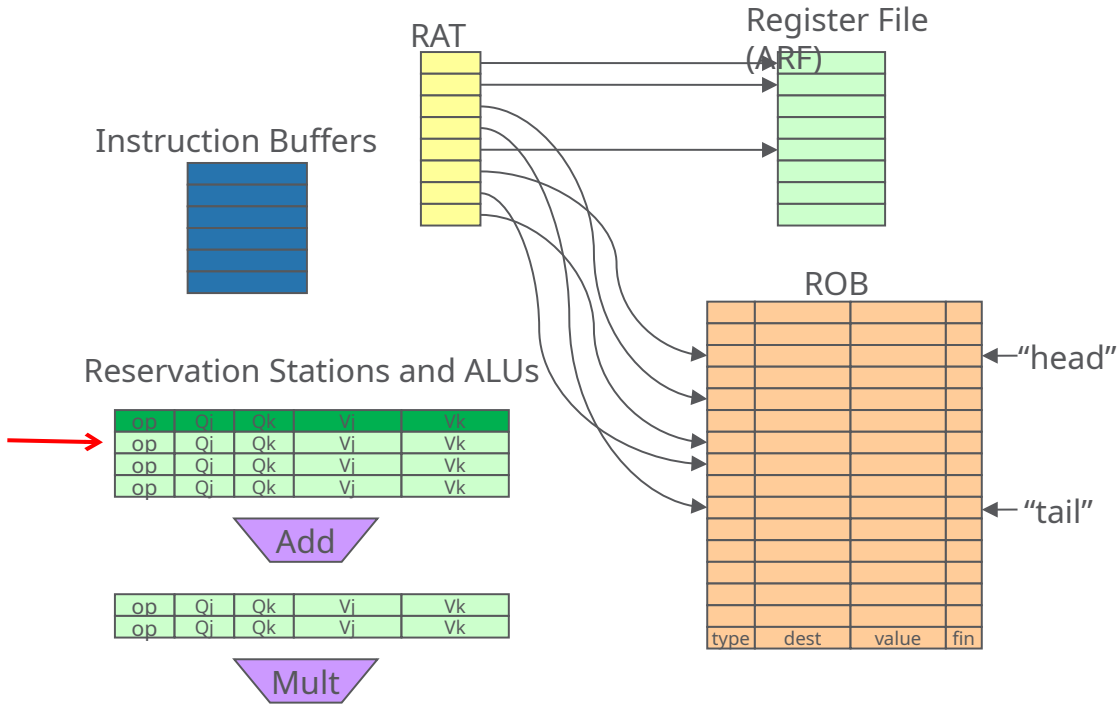
Write Result (2)

- Update ROB entry
  - The ARF holds the “official” register state, which we will only update in program order
  - Mark ready bit in ROB (to note that this instruction has completed execution)



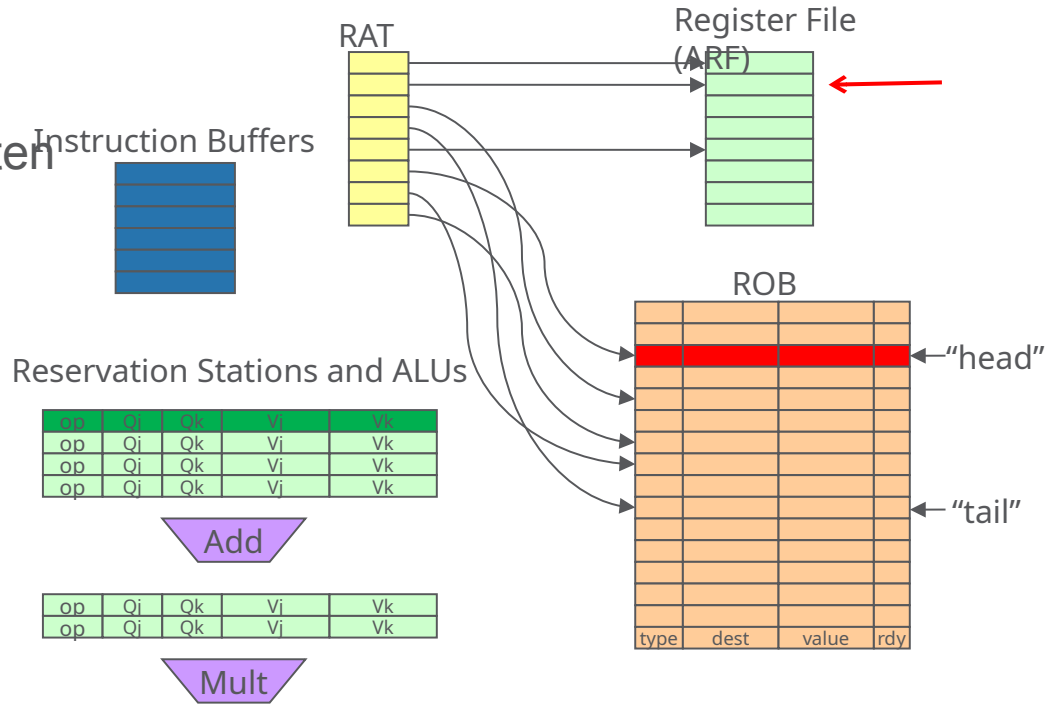
Write Result (3)

- Free the RS entry



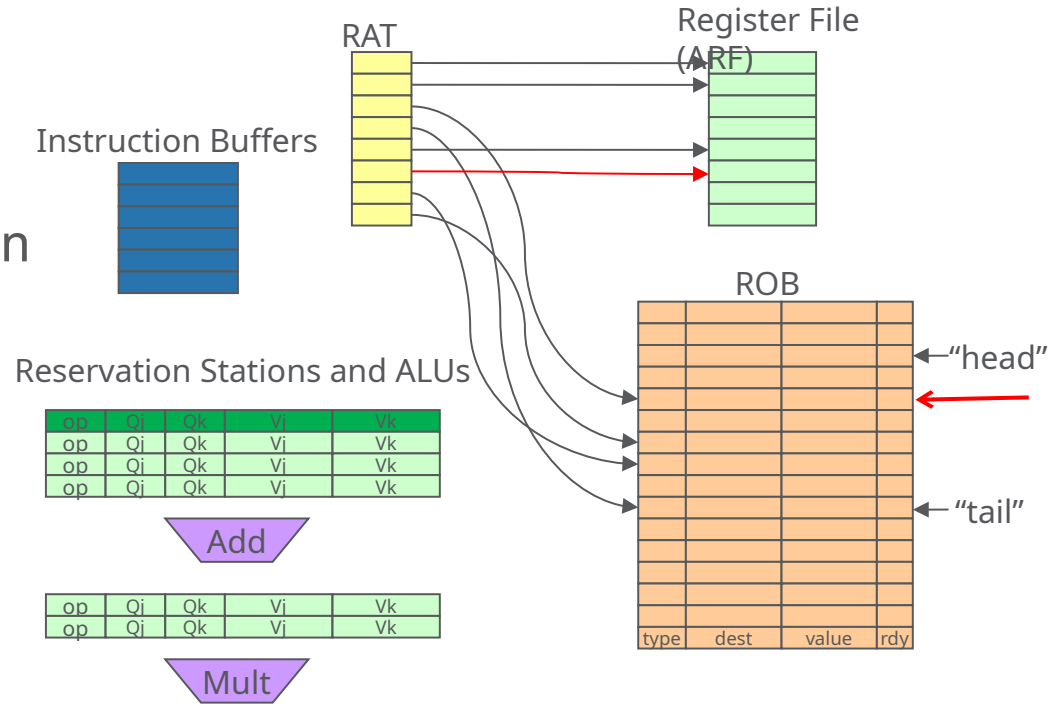
New: Commit

- Wait until ROB's head is ready
  - e.g. its result has been written
- Update Register File (ARF)
  - If instr write to register file
- Update Memory
  - If instr write to memory



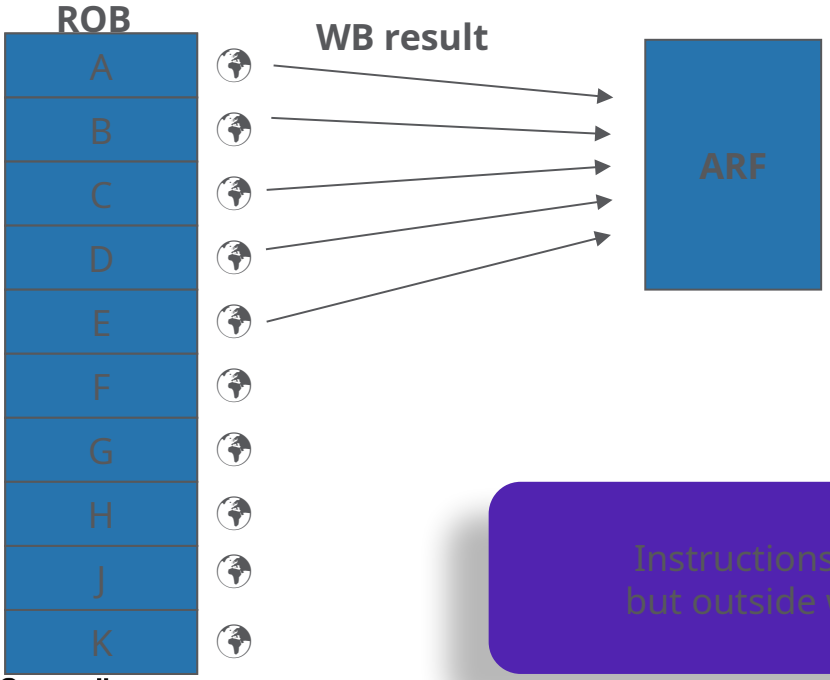
New: Commit (2)

- Update the RAT entry
  - Now points to the ARF
- Advance the ROB's head
  - to point to next instruction



Commit Illustrated

- Make instruction execution “visible” to the outside world
  - “Commit” the changes to the architected state



Outside World “sees”:

A executed

B executed

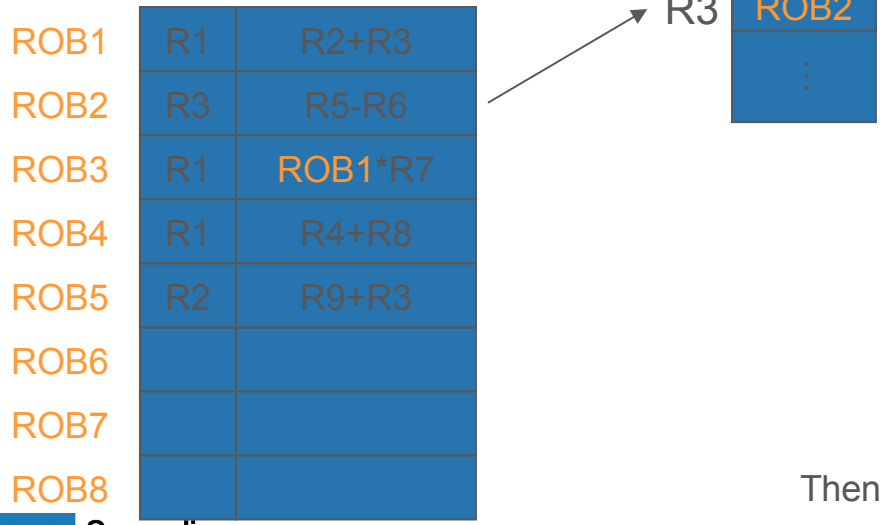
C executed

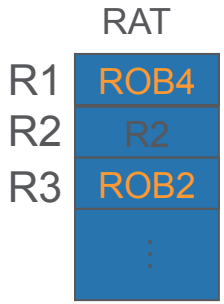
D executed

E executed

Instructions execute out of program order,
but outside world still “believes” it’s in-order

Revisiting Register Renaming





~~R1 = R2 + R3~~
~~R3 = R5 – R6~~
R1 = R1 * R7
R1 = R4 + R8
R2 = R9 + R3

If we issue R2=R9+R3 to the ROB now, R3 comes from ROB2

However, if R3=R5-R6 commits first…

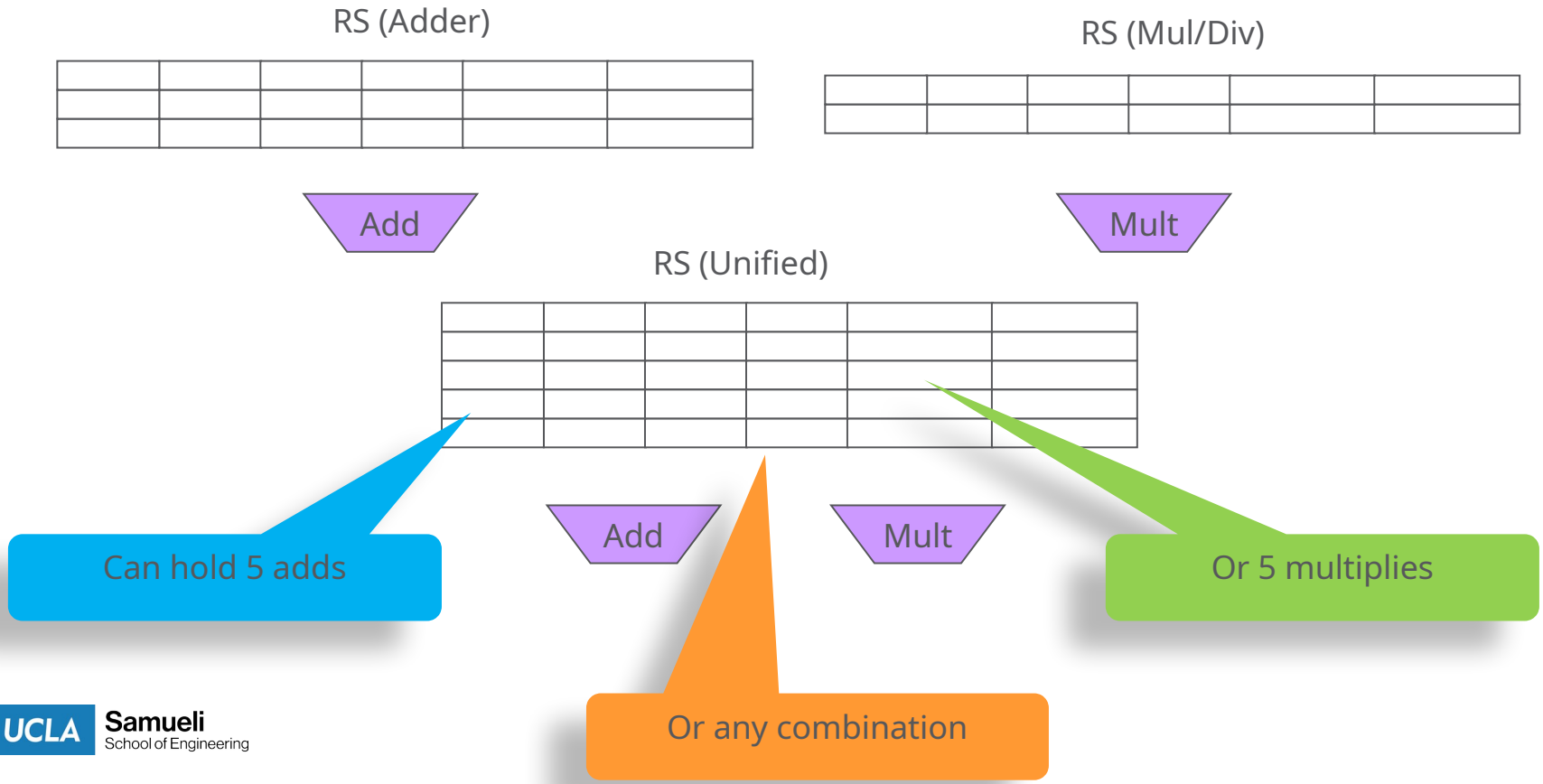
Then update RAT so when we issue R2=R9+R3, it will read source from the ARF.

Unified Reservation Stations

We stall if we need to issue a MULT when all its RS's are in use

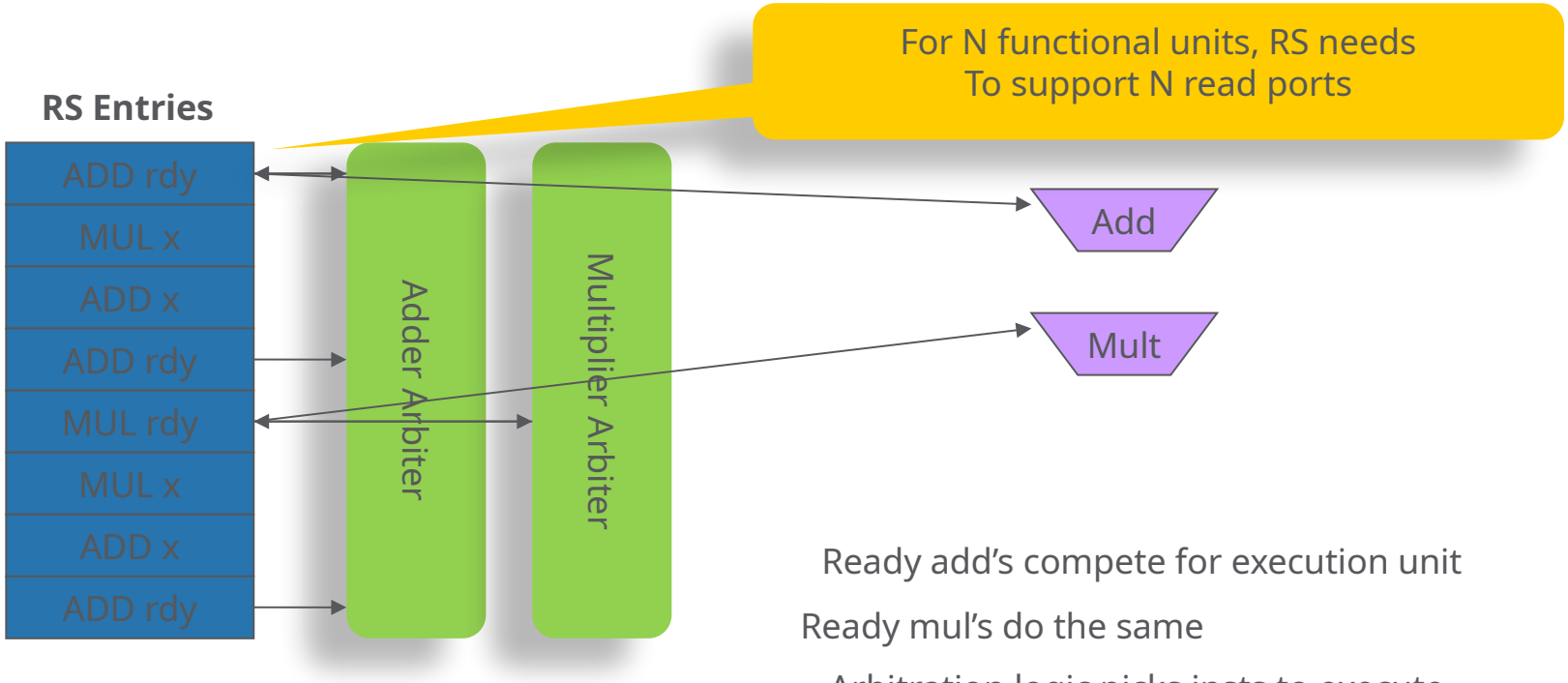
- But there may be other RS's (e.g., add) that are available
- Proper number of RS's per ALU needs to match the program's instruction distribution
  - But different programs have different distributions

Unified Reservation Stations (2)



Unified Reservation Stations (3)

- Arbitration and execution paths a little more complex (not too bad though)



Ready add's compete for execution unit

Ready mul's do the same

Arbitration logic picks insts to execute

Insts go and do their thing

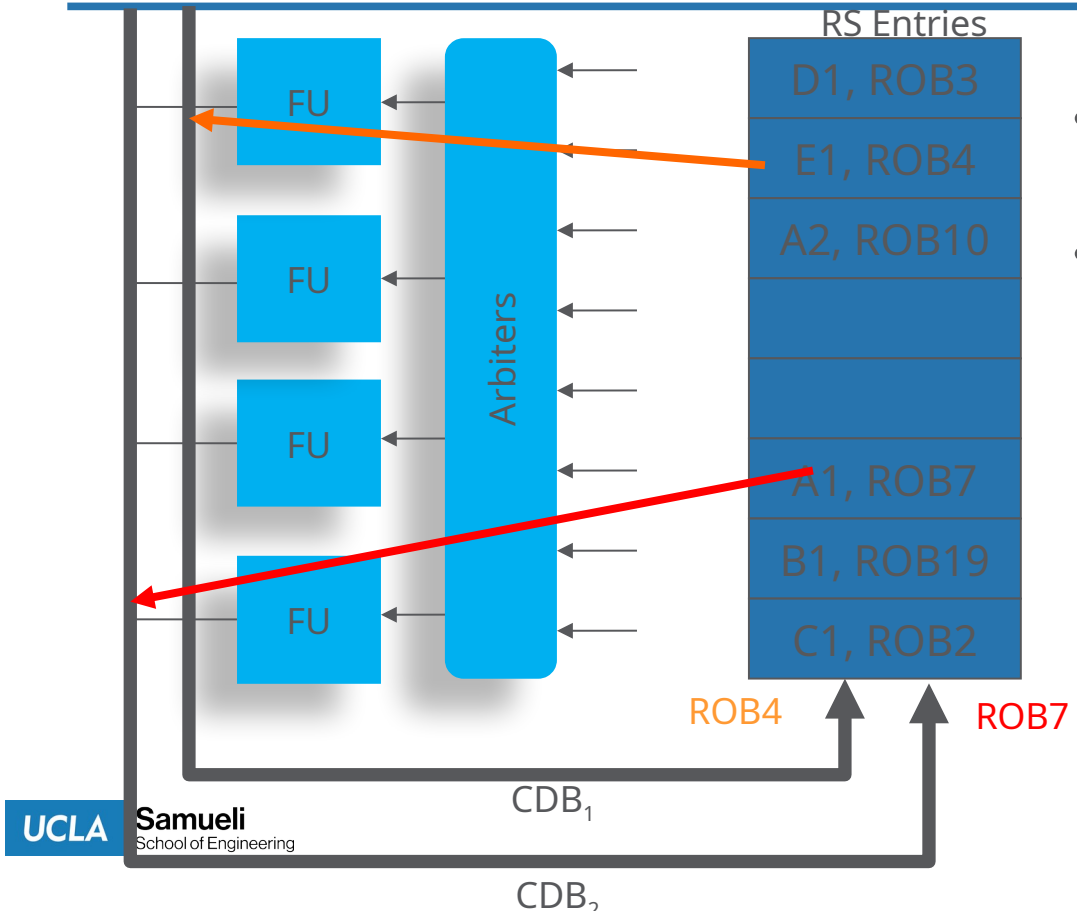
Tomasulo Drawbacks

- The Common Data Bus (CDB) bottleneck
  - Single broadcast medium
  - Sharing among FU's, register file, Store buffer
  - Contention
    - Multiple producers
  - Scalability
    - Multiple receivers
  - Long latency broadcast
- Performance limited by Common Data Bus
  - Multiple CDBs => more FU logic for parallel associative stores

Tomasulo Drawbacks

- Performance limited by Common Data Bus
- Solutions:
  - Split Buses
    - More FU's can update their CDB in parallel
  - Crossbar switch
  - Point-to-point connections
  - Hierarchical Buses

Multiple CDB's



- It works (we do this today in CPUs!)
- But there's a cost
  - each RS entry must compare each source operand with each CDB
  - more logic => more area => more power => $$$

Out-of-order, but not superscalar

- Described CPU: Tomasulo+ROB
  - CPU can only maintain sustained throughput of 1 IPC
- Needed Optimizations:
  - Need superscalar fetch, decode, etc.
  - There's only one CDB, so only one instuction per cycle can write-back its result to the ROB
  - Also must commit > 1 IPC

Getting >1 IPC

- Must be able to issue >1 IPC to RS's/ROB
- Must be able to send >1 IPC to FU's
  - Original Tomasulo can do this already (FU's are assigned instructions independently)
- Must be able to write-back >1 IPC to ROB and RS's

Dual-Issue

- Need to check resource availability for two instructions (RS's/ROB entries)
  - Depending on resources, may issue 0,1 or 2
- Read RAT/ARF/ROB for operands
  - Renaming is a little trickier (next slide)
- Update RS's/ROB entries (not too hard)

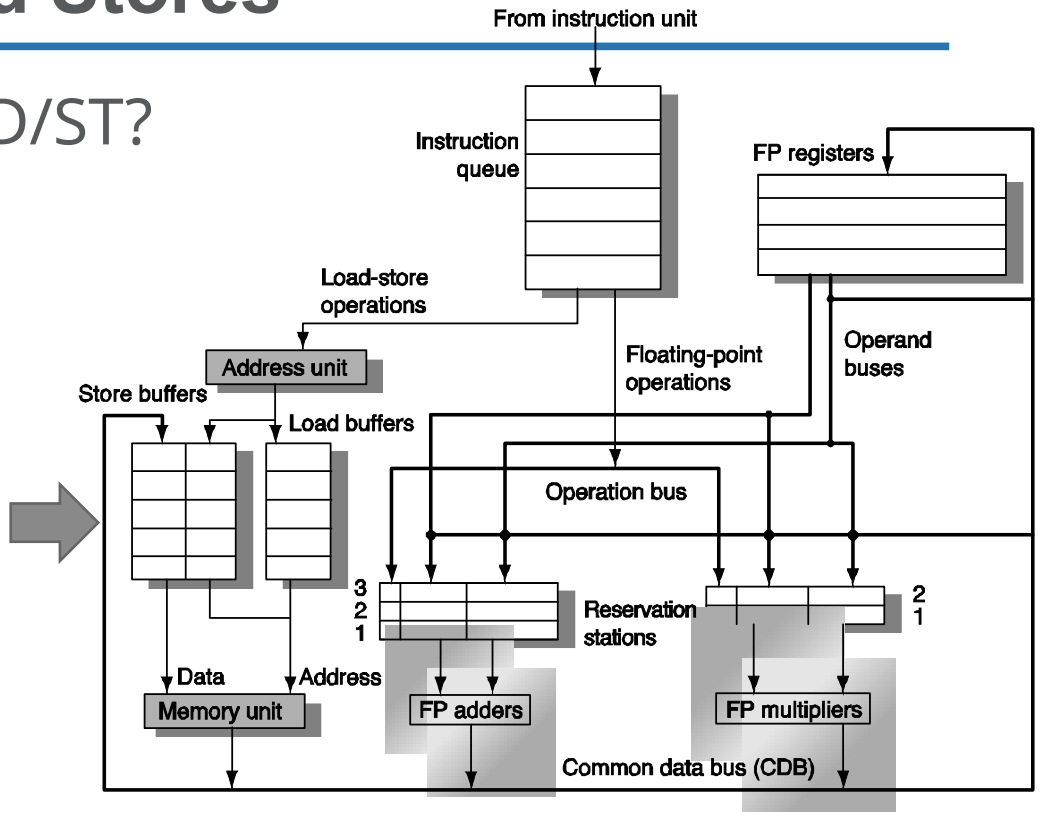
Attendance

- Please Fill the form with today’s keyword to register your attendance.

Handling Loads and Stores

What's special about LD/ST?

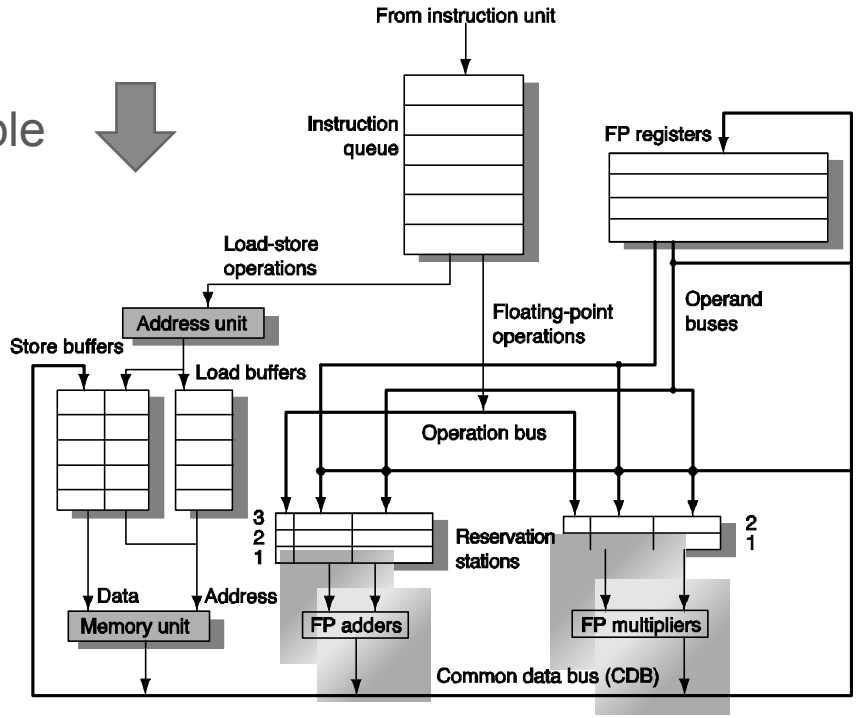
- LD latency
- ST alters memory
  - Visible to application



Handling Loads and Stores

Goal

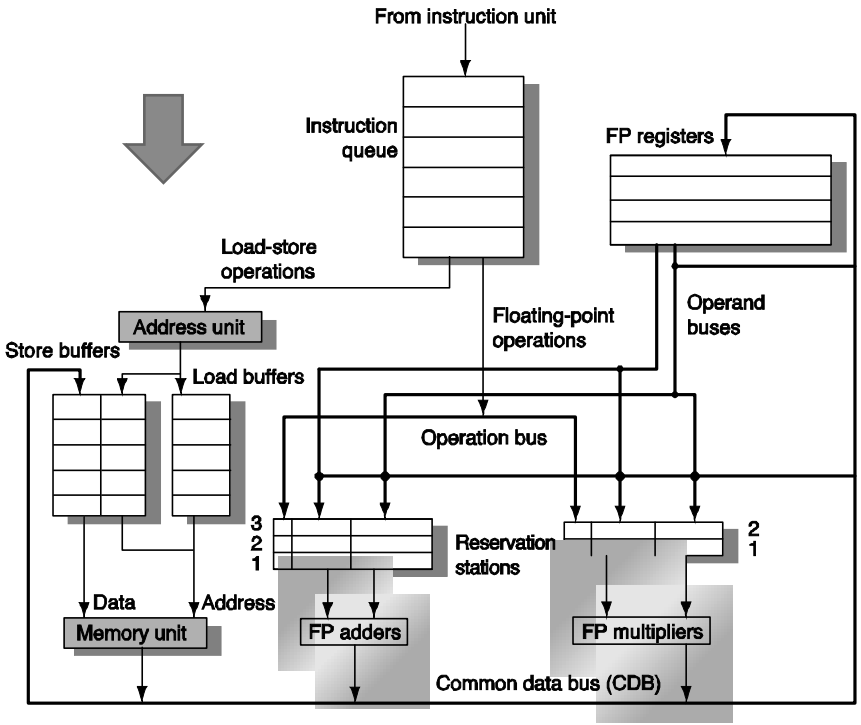
- Avoid unnecessary loads
  - Reuse already fetched data if possible
- Ensure ordered ST



Handling Loads and Stores

Solution:

- Queue LD/ST instructions
  - Similar to ROB allocation
- Update memory in-order
  - Using the ROB
- Perform address calculation
  - Can take place in the ALU pipeline or dedicated **Address Unit** (need to wait for operands)



Handling Loads and Stores

Handling Stores

- Wait for source operand
- Write value into table
- Wait for address
- Write address into table
- How are source operands obtained?

V	L/S	ADDR	DATA	A	D
1	L	104		1	0
1	S		15	0	1
1	L	204	15	1	1
0					

Handling Loads and Stores

Handling Loads

- Wait for address
- Write address into table
- Search for earlier LD/ST to avoid memory trip
- Write value into table
- Broadcast result via the common data bus (CDB)

V	L/S	ADDR	DATA	A	D
1	L	104		1	0
1	S		15	0	1
1	L	204	15	1	1
0					

Handling Loads and Stores

Load Search optimization

- Ready load with same address
  - Forward value
- Ready store with same address
  - Forward value
- Pending load with same address
  - Wait
- Pending store with unknown address
  - Wait or speculate (forward value)?
- Otherwise
  - Go to memory

V	L/S	ADDR	DATA	A	D
1	L	104		1	0
1	S		15	0	1
1	L	204	15	1	1
0					

A Brief History of OOO

- CDC 6600 (1964): scoreboard
- IBM 360/91 (1966) Tomasulo's algorithm
- Academic: Yale Patt's HPS (1985)
- Smith & Pleszkun: precise interrupt (1995)
- IBM/Motorola PowerPc 601 (1993)
- Fujitsu/HAP SPARC64(1995)
- Intel Pentium Pro (1995)

Q&A

Acknowledgement

- This course is partly inspired and use materials from the following courses created by my colleagues:
  - ECE-M1126C: Nader Sehatbakhsh @ UCLA
  - CS3220: Hyesoon Kim @ gatech